RUNI – Algorithms – Homework 5

Due date: Sunday 21/12/2025

NOTE: the general homework guidelines published in homework 1 are valid for all homework assignments in this course.

Problem 1 (24 pts):

Let $G = (V, E)$ be an undirected graph. A set $S \subseteq V$ is said to be independent if for every $u, v \in S$ they are not connected by an edge, that is $(u, v) \notin E$.

A set S is said to be **maximum** independent set if S is an independent set and largest in size. That is, if for every possible independent set $H \subseteq V$, $|S| \geq |H|$.

Assume that G is a tree.

a. Suggest a greedy algorithm that runs in linear time and finds a maximum independent set in G.

1. Initialize $S = \emptyset$.

2. While E is not empty:

    a. pick any leaf vertex v.

    b. add v to S.

    c. denote v 's parent = u . Delete v and u from V and all of u 's incident edges from E.

3. When no edges remain, add any remaining isolated vertices in V to S.

4. Return S.

b. State the GCP claim corresponding to your algorithm.

Let T be a tree, and let v be any leaf vertex of T.

There exists a maximum independent set S of T such that $v \in S$.

c. Prove the correctness of the GCP claim from item (b) for your algorithm.

Let T be a tree, and let v be any leaf vertex of T, and let u be the unique neighbor of v.

Let S^* be any maximum independent set of T.

If $v \in S^*$ then we are done.

Otherwise ($v \notin S^*$) there are 2 cases:

1. If $u \notin S^*$ then $v \cup S^*$ is an independent set of T, with more vertices than S^*. Therefore, S^* is not a maximum independent set, and we have reached a contradiction.
2. If $u \in S^*$ then following set
$$S = (S^*\{u\}) \cup \{v\}$$
Where we take S^* replace u with v, has the same number of vertices as S^*.
This new set S is also an independent set of T since v has only one neighbor, u, which is not in the new set S. Meaning S is optimal as well.

A set S is said to be **maximal** independent set if S is an independent set and for every vertex that is not in S ($u \in V, u \notin S$) the set $S \cup \{u\}$ is not an independent set.

d. Give an example of a graph $G = (V, E)$ with $V = \{a, b, c\}$, such that G has a maximal independent set, S_1, of size $|S_1| = 1$ and a maximum independent set S_2, of size $|S_2| = 2$.

$V = \{a, b, c\}, E = \{(a, b), (b, c)\}$

$S_1 = \{b\}$

$S_2 = \{a, c\}$

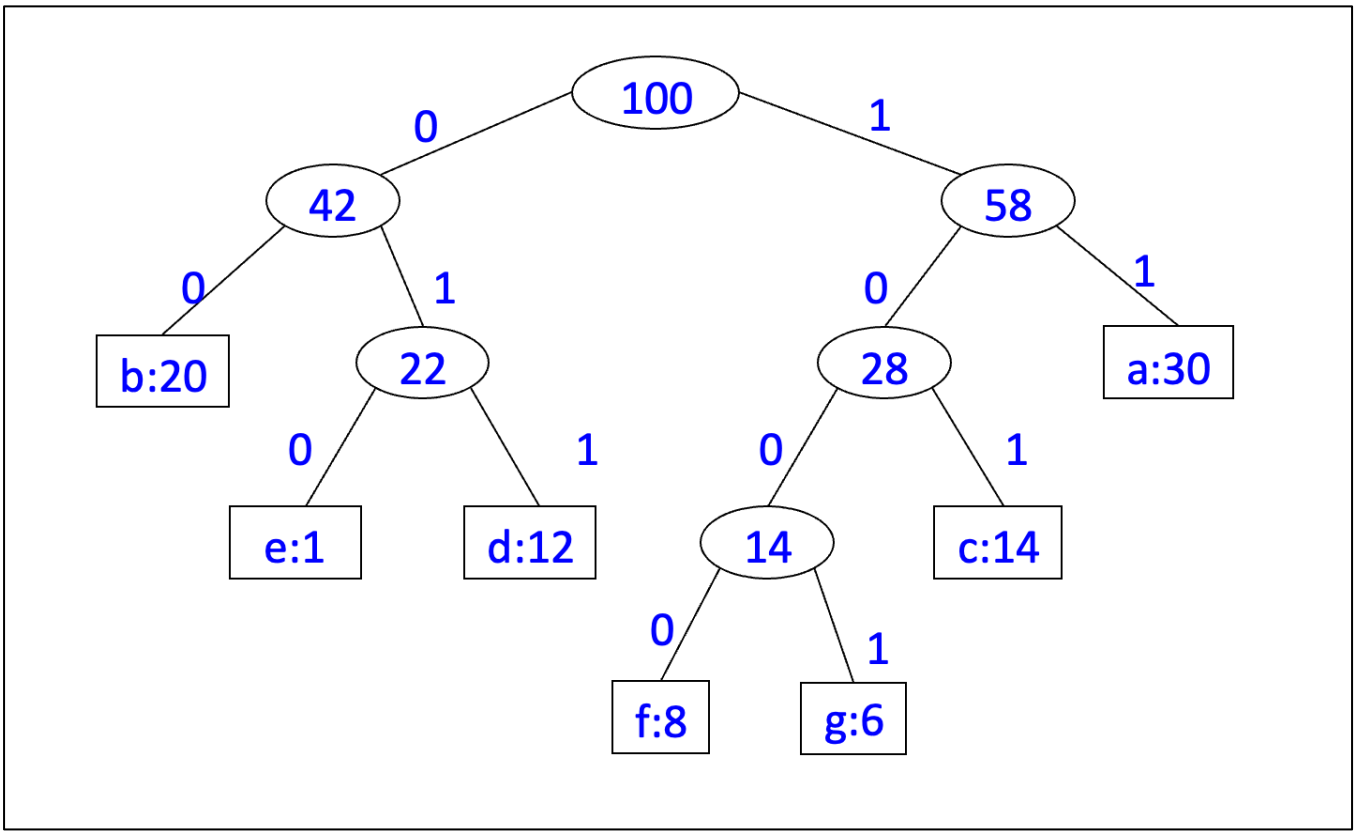
Problem 2 (10 pts):

a. Given is a text of length 100 over the alphabet $\Sigma = \{a, b, c, d, e, f, g\}$. The number of appearances of every letter in the text is given in the following table.

a	b	c	d	e	f	g
30	20	14	12	10	8	6

Find an optimal code for the text.

1. Describe the binary tree that corresponds to the code



2. Complete the table with the code of each letter.

a	b	c	d	e	f	g
11	00	101	011	010	1000	1001

Problem 3 (18 pts):

Prove or give a counter example for each of the following claims.

a. If all letters have the same frequency, then Huffman algorithm produces a fixed-length code.

False.

A counter example is with any number of letters (other than 1) that is not a power of 2.

For example $\Sigma = \{a, b, c\}$, with $p(a) = p(b) = p(c) = 1/3$.

Huffman may produce $a = 00, b = 01, c = 1$, which is not a fixed-length code.

b. If for two letters $a, b \in \Sigma$ it holds that $p(a) \leq p(b)$ then in any optimal Huffman coding given by a tree T, $d_T(a) \leq d_T(b)$ (the depth of a in T is equal to or smaller than the depth of b in T).

False.

Repeat the counter example from item (a).

$\Sigma = \{a, b, c\}$, with $p(a) = p(b) = p(c) = 1/3$.

Huffman may produce $a = 00, b = 01, c = 1$, for which $p(b) \leq p(c)$ but $d_T(b) > d_T(c)$.

c. If Huffman produces a fixed length code then for any two letters $a, b \in \Sigma$, $\frac{1}{2} \leq p(a)/p(b) \leq 2$.

False.

For example $\Sigma = \{a, b\}$, with $p(a) = 0.9, p(b) = 0.1$.

Huffman produces $a = 0, b = 1$, but $p(a)/p(b) = 9 > 2$.

Problem 4 (24 pts):

As the cold winter season begins, heavy rains start pouring on the Negev desert. Despite preparations, severe floods catch Netivei Israel (National Transport Infrastructure Company) by surprise. An urgent request goes out for a plan to position aid teams along the country's main southern artery.

Route 90, the longest road in Israel, is now a top priority. Along the portion stretching from the Dead Sea to Eilat, n flood events have already been recorded. Each flood event is associated with a segment $[s_i, t_i]$ representing the kilometer interval of the road that may have been affected. Note that a point on the road may unfortunately belong to multiple such segments.

An emergency team can be parked at any kilometer point x along the road. A team parked at x can effectively handle any flood prone segment $[s_i, t_i]$ that contains x ($s_i \leq x \leq t_i$).

Netivei Israel company seeks to deploy as few teams as possible, while still ensuring that every flood-prone segment contains at least one team.

a. Propose an algorithm that, given the set of segments $\{[s_i, t_i]\}_{i=1}^{n}$, determines where to place teams in order to minimize the total number of teams required.

Algorithm:

1. Sort the segments so that $t_1 \leq t_2 \leq \cdots \leq t_n$.

2. Initialize X, the set of team positions.

3. Initialize $last = 0$.

4. for i from 1 to n:

    a. If $s_i > last$:

        (1) place a team at position t_i.

        (2) add t_i to X.

        (3) update $last = t_i$.

5. Return X.

b. Prove that your algorithm is optimal.

Let the segments be sorted so that $t_1 \leq t_2 \leq \cdots \leq t_n$.

The GCP: The exists an optimal solution that places a team at t_1.

GCP Proof:

Let X^* be an optimal solution. If $t_1 \in X$ we are done.

Otherwise, since X^* is a valid solution so it must have a team at some point $s_1 \leq p \leq t_1$.

Define $X = (X^* \backslash \{p\}) \cup \{t_1\}$.

That is, take X^* but remove p and replace it with t_1. Clearly $|X| = |X^*|$.

Consider a segment $[a, b]$. If it did not contain p then it contained another point in X^* and it is still covered. If $[a, b]$ did contain p then we show that $t_1 \in [a, b]$. Assume towards contradiction that $t_1 \notin [a, b]$, but then because $p \leq t_1$ it must be that $t_1 > b$ which contradicts t_1 being the smallest right endpoint.

Thus, X is a valid solution, optimal in size.

Algorithm proof: We show by induction on the number of segments S that the greedy algorithm is optimal.

Base case: $|S| = 0$. No flooded segments and no teams needed. The algorithm is correct.

Inductive step: Assume the algorithm is optimal for all instances with fewer than k segments.

By the GCP, there exists an optimal solution that places a team in t_1. The algorithm places a team there, and can ignore all segments covered by t_1. By the induction hypothesis the algorithm will solve optimally for the remaining segments. Thus, the algorithm is optimal for $|S| = k$ segments.

c. Analyze your algorithm's runtime complexity.

Sorting the segments is done in $O(n \cdot \log n)$.

The single scan performs $O(1)$ work per segment, for a total of $O(n)$ time.

The total runtime is dominated by the sorting, for a total of $O(n \cdot \log n)$.

Problem 5 (24 pts):

A music festival has two stages running simultaneously: the **Rock Stage** and the **Jazz Stage**. There are $N_R>0$ seats available at the Rock Stage, and $N_J>0$ seats available at the Jazz Stage, with $N_R+N_J=n$.

n people arrive at the festival. Every person i gives an integer enjoyment score r_i if they attend Rock and j_i if they attend Jazz. Offer the festival manager an efficient greedy algorithm that divides the attendees into two groups R and J, so that $|R| = N_R$, $|J| = N_J$, group R will be directed to the Rock stage, group J will be directed to the Jazz stage, and their total satisfaction, given by $\sum_{i\in R} r_i + \sum_{i\in J} j_i$ will be maximal.

1. Describe the algorithm. State and justify its time complexity.
2. State and prove the corresponding greedy choice property.

Algorithm:

1. For every person, compute the difference $d_i = r_i - j_i$ (may be negative)
2. Sort the differences so that $d_1 \geq d_2 \geq \cdots \geq d_n$
3. Let R be the first N_R people according to this order
4. Let J be the remaining $n - N_R$ people

Runtime analysis: step (1) takes $O(n)$ time; step (2) requires sorting so it takes $O(n \cdot \log n)$ time; steps (3)-(5) take $O(n)$ time. Runtime is dominated by the sorting step for a total of $O(n \cdot \log n)$ time.

The GCP: There exists an optimal solution in which the tourist with maximal difference is in R.

Proof [exchange argument]:

Consider a solution in which attendee 1 is in J.

Replace her with any other attendee j in R. The change in the total satisfaction is $r_1 - j_1 - r_j + j_j$.

Since $r_1 - j_1$ ^{3} $r_j - j_j$, the change is non-negative.